

Engenharia de Software 2

(ES)

Universidade Federal do Piauí

Prof. Leonardo Vasconcelos
leonardo.vasconcelos@ufpi.edu.br



Prova dia 28/04



**Não teremos aula presencial no dia
23/04**

**O tempo será disponibilizado para
revisão conforme votado em turma.**



Na aula anterior ...

- Organização dos Trabalhos



Hoje



Trabalho 2 - Parte 1



Trabalho 2

- Parte 1: Elaborar uma palestra com os temas definidos
 - 1 ou 2 pessoas
 - Prazo (23/04)
 - Quem não escolher no prazo, terá o seu nome jogado aleatoriamente em algum dia.
 - Entrega: 1 dia antes da apresentação (será descontado 1 ponto de quem não entregar)
 - 45 minutos
 - Quem faltar fará uma prova



Trabalho 2

- Parte 1: Elaborar uma palestra com os temas definidos
 - Não poderá ter temas repetidos. O dono do tema será o que escolher primeiro.
 - No dia da apresentação, todos deverão chegar às 8:00 em ponto. Quem não chegar, obrigatoriamente fará a prova
 - A apresentação deverá conter
 - Conceitos básicos
 - Um exemplo de código (implementação)
 - Exemplos de aplicação
 - Dúvidas do trabalho serão tiradas até 3 dias antes da entrega
 - Entregar o material e as referências (artigo científico, livro, documentação de software)
 - Enviar no Sigaa e Grupo do Whats



Trabalho 2

- Erros comuns
 - Fazer uma implementação sem profundidade
 - “Encher linguiça” - Será altamente penalizado!
 - Usar o tempo explicando conceitos fora do tema proposto
 - Não usar o tempo de forma correta
 - Passar do tempo de Apresentação estipulado
 - Apresentar menos do Tempo estipulado
 - **Nao colocar uma imaem antes de explicar o código**



Trabalho 2

- Temas:

- 1 - Ciclo de Vida de Injeção de Dependência Mylena e Rais
- 2 - Padrões de Projeto: Mediator, Responsibility, Observability Kaylane e Geovana
- 3 - Padrões de Projeto: Command Query Responsibility Segregation. Maria Julia e Lucrecia
- 4 - Domain-Driven Design (DDD). Alysson e Vinicius
- 5 - Inversão de Controle (IoC) e Contêineres de Injeção de Dependência. Antonio Mauricio e Rogers
- 6 - Padrões de Resiliência (ex: Circuit Breaker, Retry, Bulkhead). Emerson e Carlos Brito
- 7 - Event Sourcing vs. State Transfer Breno e Diogo

Trabalho 2

- Temas:

- 8 - Arquitetura Hexagonal (Ports and Adapters). Sara e Carlos Eduardo
9. Mensageria: Kafka, RabbitMQ. Filipe e guilherme
10. Mensageria: Azure Service Bus. Nicolas e Pedro Kauan
11. Integração e Entrega Contínua (CI/CD). Implementar o conceito já foi explicado em aula (falar comigo quem escolher esse tema) Gabriel Oliveira
12. Serverless Architectures. Ryan e grazielly
13. Arquitetura de Microserviço (Falar com o professor).
14. Inteligência Artificial no Desenvolvimento de Software – como usar IA para desenvolver sistemas, auxiliar em testes, geração de código e documentação (Falar com o professor). Alison e Wagner

Trabalho 2

- Datas de apresentação (2 por dia)
 - 05/05 -
 - 07/05 -
 - Ciclo de Vida de Injeção de Dependência Mylena e Rais
 - 12/05 -
 - Padrões de Projeto: Mediator, Responsibility, Observability Kaylane e geovana
 - Mensageria: Kafka, RabbitMQ. Filipe e guilherme
 - 14/05 -
 - Padrões de Projeto: Command Query Responsibility Segregation. Maria Julia e Lucrecia
 - 8 - Arquitetura Hexagonal (Ports and Adapters). Sara e Carlos Eduardo



Trabalho 2

- Datas de apresentação (2 por dia)
 - 19/05 -
 - 5 - Inversão de Controle (IoC) e Contêineres de Injeção de Dependência. Antonio Mauricio e Rogers
 - 6 - Padrões de Resiliência (ex: Circuit Breaker, Retry, Bulkhead). Emerson e Carlos Brito
 - 21/05 -
 - Mensageria: Azure Service Bus. Nicolas e Pedro Kauan
 - 12. Serverless Architectures. Ryan e grazielly
 - 26/05 -
 - 7 - Event Sourcing vs. State Transfer Breno e Diogo
 - 11. Integração e Entrega Contínua (CI/CD). Implementar o conceito já foi explicado em aula (falar comigo quem escolher esse tema) Gabriel Oliveira
 - 28/05 -
 - 4 - Domain-Driven Design (DDD). Alysson e Vinicius
 - 14. Inteligência Artificial no Desenvolvimento de Software – como usar IA para desenvolver sistemas, auxiliar em testes, geração de código e documentação (Falar com o professor). Alison e Wagner

Tipos de Arquitetura



Objetivos



Objetivos

- Oferecer uma base sobre as principais arquiteturas utilizadas no desenvolvimento de sistemas.
- Objetivos Secundários
 - Análise de Casos Práticos
 - Aplicação Prática

Introdução



Introdução

- Diferentes arquiteturas foram desenvolvidas ao longo do tempo
 - Algumas com prática, outras com teoria
 - Elas ensinam como dividir, como conectar, como trocar informação

Tipos de Arquitetura



Arquitetura de Sistemas

- Seis grandes abordagens
 - Baseada em camadas
 - Orientada a objetos
 - Baseada em eventos
 - Centrada em dados
 - Arquitetura cliente/servidor
 - Arquitetura P2P

Arquitetura de Sistemas

- Cada qual com vantagens/desvantagens
- Não necessariamente estritas
 - grandes sistemas reais misturam as abordagens
- A arquitetura pode mudar ao longo do tempo

Arquitetura em Camadas

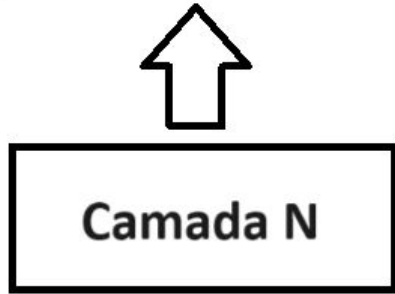


Arquitetura em Camadas

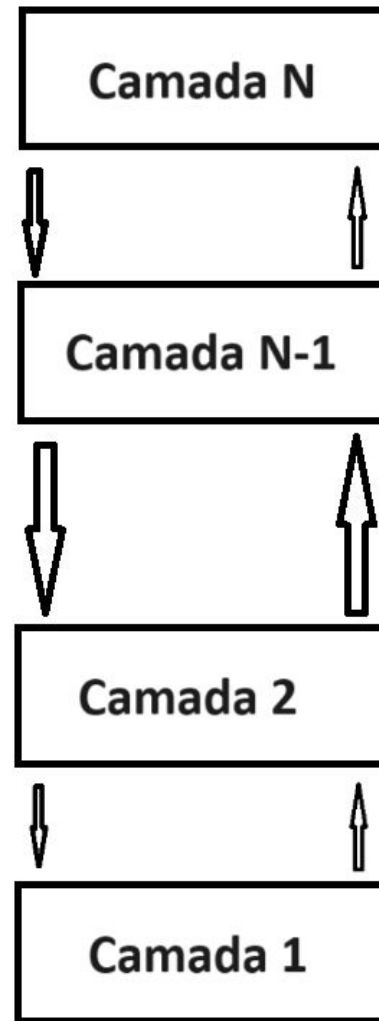
- Componentes organizados “verticalmente”
 - facilita organização e modificação
- Camada implementa um serviço (função)
 - utiliza serviço da camada inferior
 - oferece serviço a camada superior
- Possibilidade de trabalhar com times diferentes

Arquitetura em Camadas

Oferece serviços
para a camada $x+1$

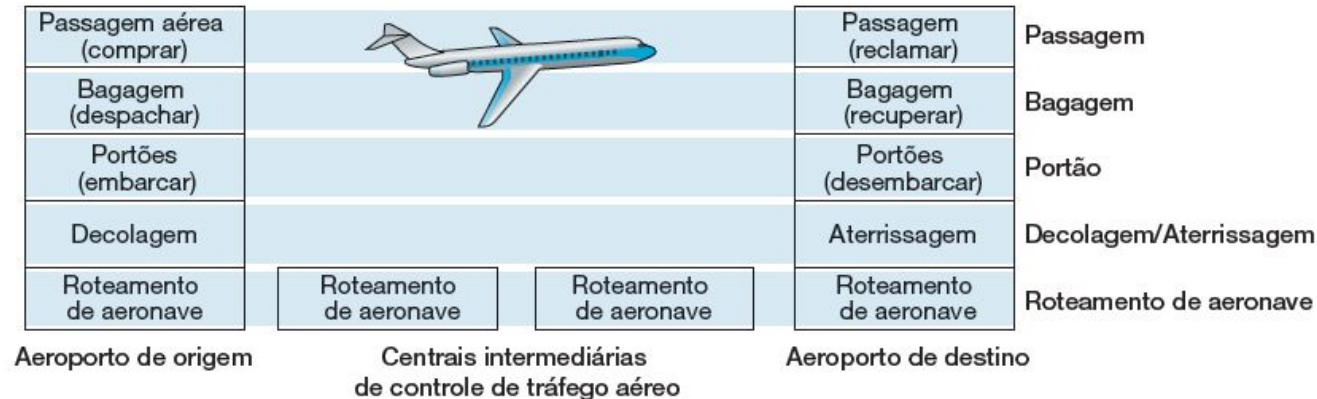


Utiliza serviços da
camada $x-1$



Arquitetura em Camadas

- Exemplos
 - Pilha de protocolos TCP/IP
 - Sistemas Operacionais
 - Sistemas de Reservas em Linhas Aéreas



Arquitetura em Camadas

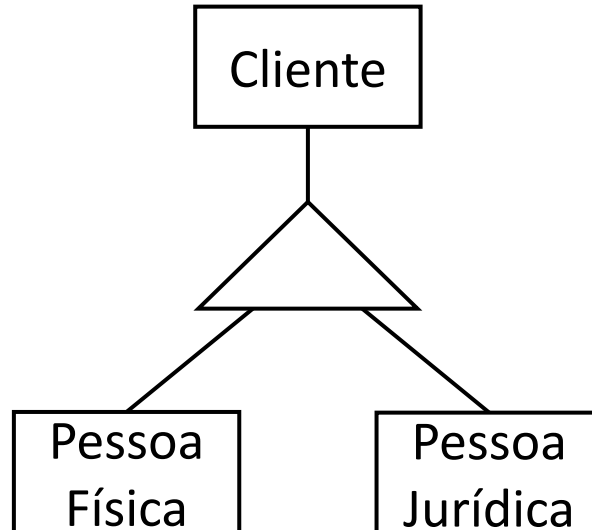
- Cliente, servidor e banco de dados
- MVC

Arquitetura Orientada a Objetos



Arquitetura Orientada a Objetos

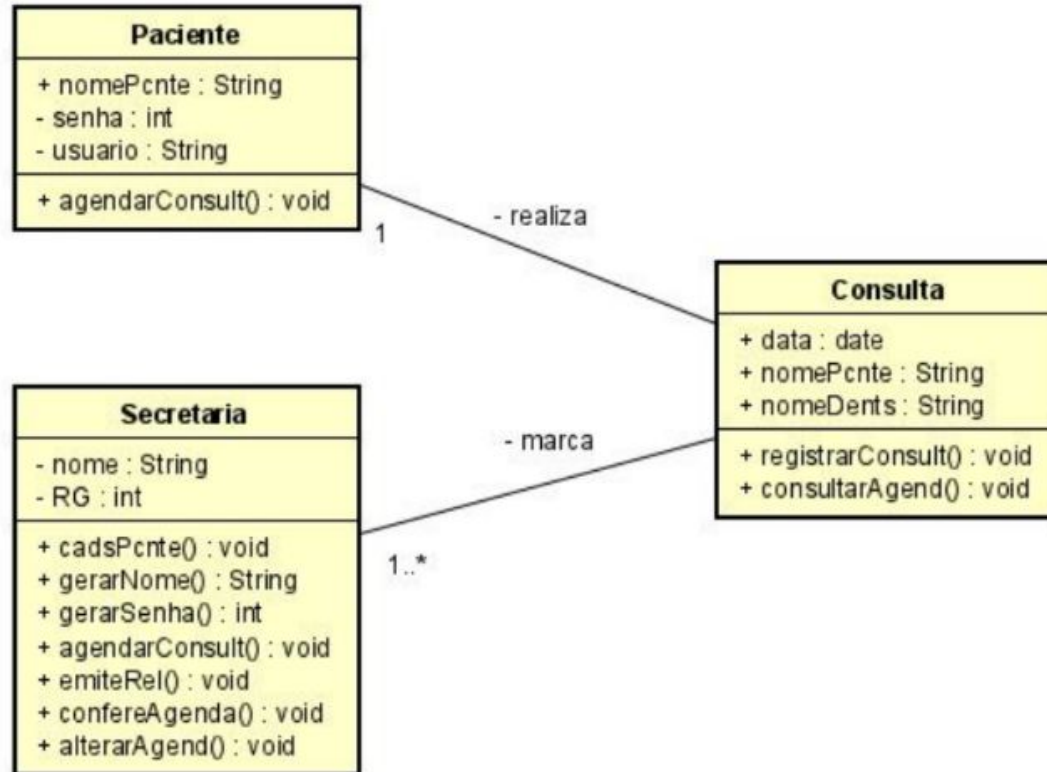
- Usam conceitos de Orientação a Objeto
 - herança, polimorfismo, encapsulamento



Arquitetura Orientada a Objetos

- É facilmente visualizado por um diagrama
- Objetos implementam funcionalidades
 - utilizam serviços de outros objetos
 - chamam objetos necessários

Arquitetura Orientada a Objetos



Arquitetura Orientada a Objetos

- Exemplo
 - Qualquer sistema que implementa a orientação a objetos

Arquitetura em Eventos



Arquitetura em Eventos

- Event Driven Architecture
- Componentes organizados em um “barramento” (também chamado de broker)
 - maior simplicidade
- Resiliente x não tem memória de longo prazo

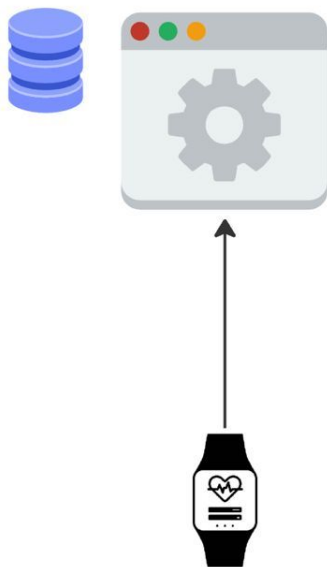
Arquitetura em Eventos

- Componentes enviam/recebem eventos
 - eventos tem identidade e informação, processados apenas onde necessário
 - utilizam serviço de outros componentes
 - barramento não tem memória (de longo prazo)

Arquitetura em Eventos

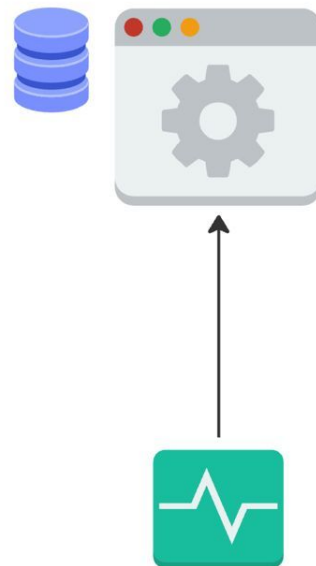
- Exemplo
 - Publish/Subscribe (assinantes)
 - Twitter
 - Semáforo inteligente (sensor)
 - Detectar acúmulo de pessoas
 - Quando você recebe um arquivo no WhatsApp

Arquitetura em Eventos



- Registra eventos
- Traz infos quantitativas

BROKER



- Faz comparações com outros dados
- Faz monitoramento e envia alertas

Arquitetura Centrada em Dados

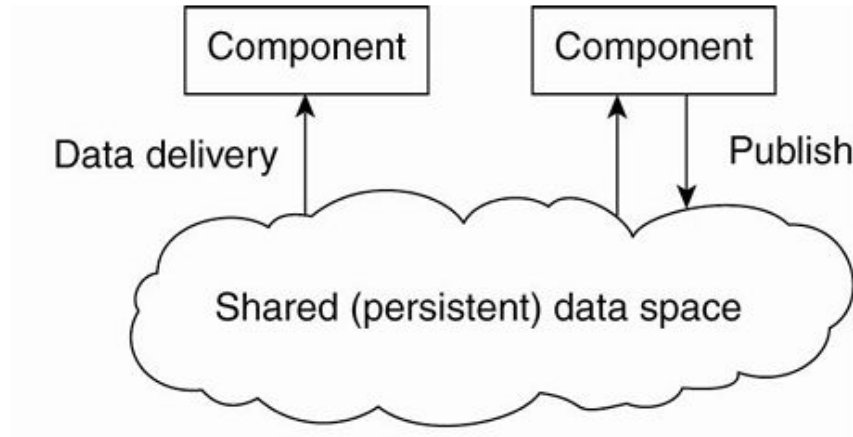


Arquitetura Centrada em Dados

- Componentes usam um repositório de dados compartilhado
 - comunicação indireta, através repositório
 - repositório registra (armazena) toda informação
 - repositório é um outro sistema

Arquitetura Centrada em Dados

- Exemplo:
 - Sistema de login



Exemplo

- Aplicativos de entrega (iFood, Uber Eats)
 - O sistema coleta dados de localização, tempo de entrega, preferências de pedido, avaliações.
 - Esses dados são usados para:
 - Sugerir restaurantes
 - Otimizar rotas
 - Prever tempo de entrega
 - Personalizar recomendações

Exemplo

- Aplicativo de banco (Nubank, PicPay)
 - Analisa seus gastos, horários de uso, locais de transação.
 - Com isso:
 - Bloqueia movimentações suspeitas
 - Gera relatórios automáticos
 - Sugere investimentos com base no perfil

Arquitetura Cliente / Servidor



Cliente / Servidor

- É um tipo de arquitetura em Camada
- Cada componente assume um de dois papéis distintos: servidor ou cliente
- Papel cliente/servidor são bem distintos
- É possível separar um time para trabalhar no cliente e outro no servidor.
- Rastreável

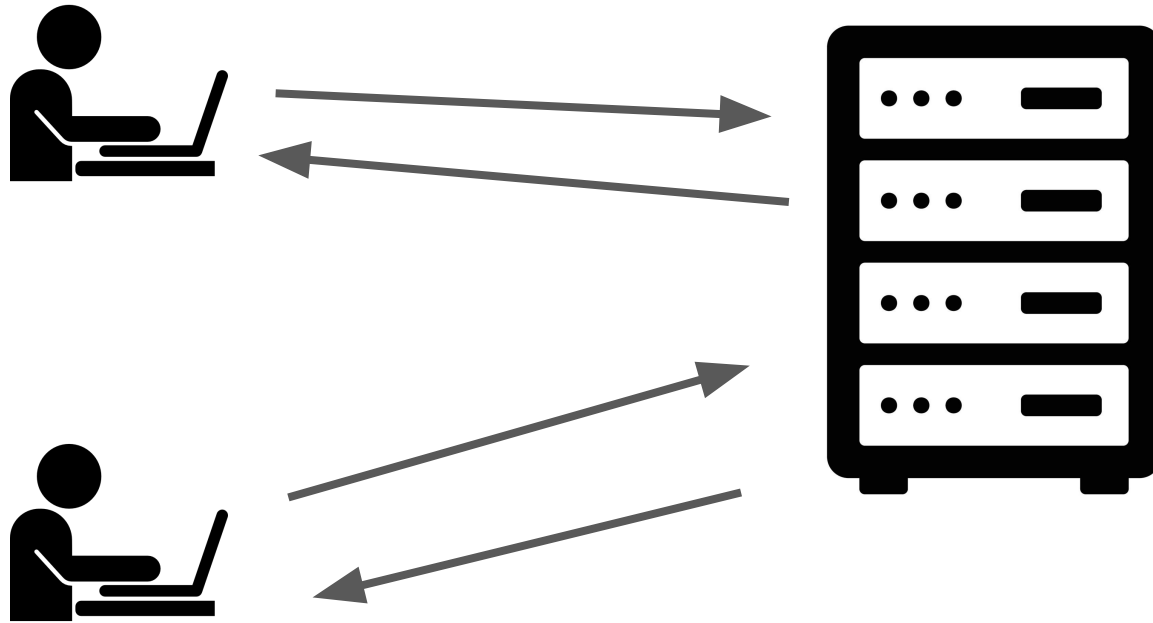
Cliente / Servidor

- Servidor: oferece um serviço
 - aguarda conexão de um cliente
 - recebe e processa pedido do cliente
 - retorna resultado
- Cliente: demanda um serviço
 - se conecta ao servidor
 - envia pedido
 - aguarda resposta

Cliente / Servidor

- Quantidade de acessos
 - Será preciso dimensionar a quantidade de servidores

Cliente / Servidor



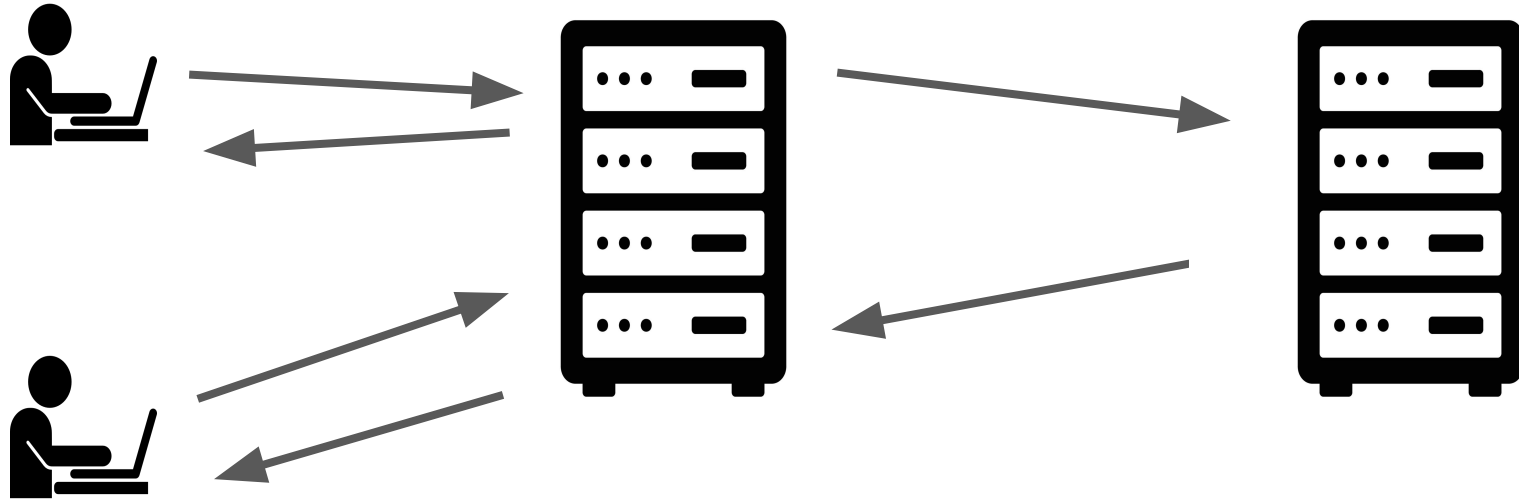
Cliente / Servidor

- Exemplo
 - Qualquer aplicação que usa o protocolo http

**Um servidor pode também
fazer papel de cliente?**

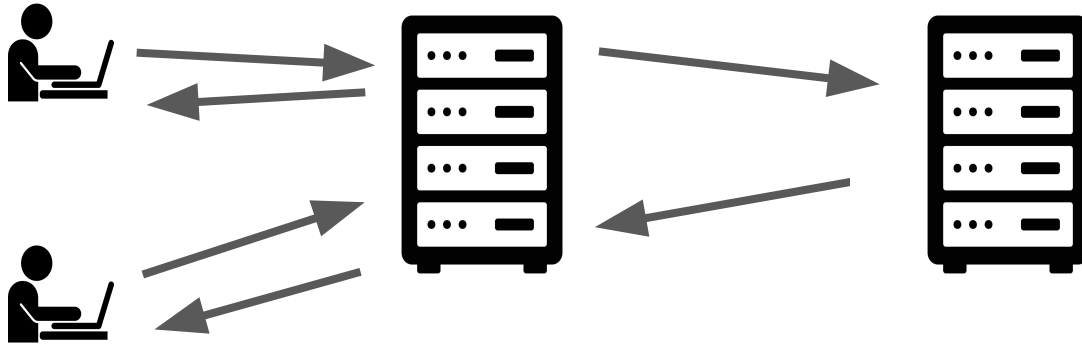


Cliente / Servidor



Cliente / Servidor

- Claro! Servidor é meu e projeto como quiser!



- Servidor 1 contacta Servidor 2 (se passando como cliente) para exercer sua função Sistema de dois níveis (two-tier system) Ex: servidor web, autenticação, banco de dados

Arquitetura P2P



Arquitetura P2P

- Um modelo de rede em que os participantes têm funções semelhantes e podem agir tanto como clientes quanto como servidores.
- Informação compartilhada
- É resiliente
- Cada componente pode atuar como cliente ou servidor

Exercícios



Exercícios

1 - Escolha um grupo de 3 pessoas e discuta exemplos de aplicações (não apresentadas em aula) que possam utilizar as arquiteturas:

- baseada em camadas:
- baseada em orientação a objetos
- baseada em eventos
- centrada em dados
- arquitetura cliente/servidor
- arquitetura P2P

Exercícios

2 - Escolha um grupo de 3 pessoas e desenvolva uma aplicação com o conhecimento que você tem até o momento. Crie um padrão de arquitetura e explique-o detalhadamente.

Referências

- Pressman, R. S., & Maxim, B. R. (2021). Engenharia de software-9. McGraw Hill Brasil.
- Ian, S. (2011). Engenharia de software. 6a. edição, Addison-Wesley/Pearson.
- Kurose, J. F., & Ross, K. W. (2006). Redes de Computadores e a Internet. São Paulo: Person, 28.